Data Science for Economists

Lecture 9: Intro to Data Science and Model Fitting

Jiaxi Li
University of North Carolina | ECON 370

Table of contents

Data Science

Motivation

We've spent most of the class learning R and building solid programming skills.

Today and next few lectures, we'll take a quick tour of the broader data science world and explore how R is used in real workplace settings.

We'll be covering many concepts and models at a **high level**—you don't need to master them all right now.

The goal is to give you a taste of what's out there, so that when you study these topics in depth later, you'll already understand the intuition and know how to start coding them.

What is data science?

Data science is a relatively new "field" that is still evolving.

Wikipedia's definition:

> Data science is an interdisciplinary field that uses scientific methods, processes, algorithms and systems to extract knowledge and insights from noisy, structured and unstructured data, and apply knowledge and actionable insights from data across a broad range of application domains. Data science is related to data mining, machine learning and big data.

Another definition:

> Data Scientist (n.): Person who is better at statistics than any software engineer and better at software engineering than any statistician. - Josh Wills

Anyone can be a data scientist!

- Economists have a special toolkit that is more important for data science than ever.

What do data scientists do?

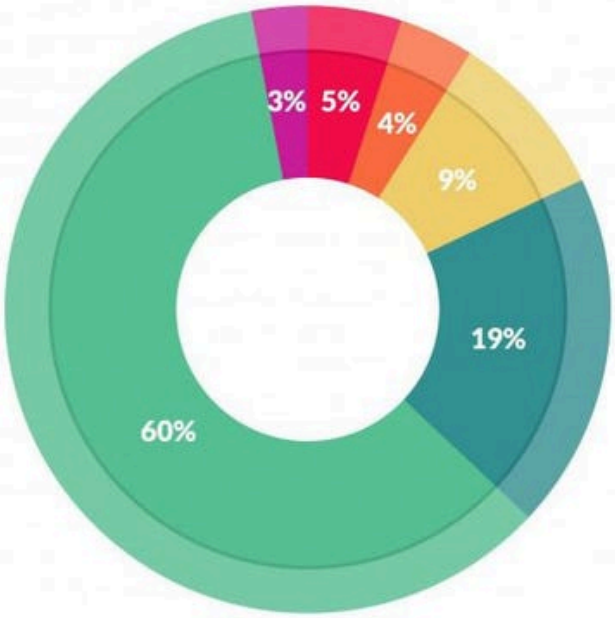
1. **Data mining**: extracting, wrangling, and storing large amounts of data.
2. **Modeling**: applying models and ideas from both statistical/machine learning and traditional statistics to build algorithms to do things too difficult for humans.
3. **Software/website development**: some data scientists will take the data, algorithms, and insights they develop and integrated them into software or websites.

There are lots of buzzwords in this area. It is important to see through this and not get intimidated.

We've already spent a bit of time talking about data wrangling.

Let's talking about the modeling side.

What do data scientist do?



What data scientists spend the most time doing

- *Building training sets: 3%*
- *Cleaning and organizing data: 60%*
- *Collecting data sets; 19%*
- *Mining data for patterns: 9%*
- *Refining algorithms: 4%*
- *Other: 5%*

Modeling

AI, Machine Learning and Deep Learning

In this new era, we have been bombarded with terms such as AI, ML and DL.

- **Artificial Intelligence (AI)**: The broad field of creating systems that mimic human intelligence.
- **Machine Learning (ML)**: A subset of AI that uses data-driven algorithms to learn patterns and make predictions.
- **Deep Learning (DL)**: A subset of ML that uses neural networks to automatically extract features, reducing the need for human intervention.

You can check this IBM article for further details.

Statistics vs Machine Learning

One of the most confusing thing more many people is understanding the difference between **machine learning** and **statistical learning**.

In truth, there isn't much of a difference and there's *lots* of overlap between the two.

- E.g. Ridge and LASSO regression are used in both and many statistical learning algorithms are just "flexible estimation" (nonparametric estimation) in statistics.

The big difference comes down to philosophical differences and objects of interest.

Suppose you have a traditional linear model:

$$y_i = x_i\beta + \varepsilon_i$$

- Statisticians will care about getting an estimate for β, called $\hat{\beta}$
  - Ideally, $\hat{\beta}$ will have desirable properties.
- Machine learning cares about getting accurate and "precise" estimates of y_i, called $\hat{y}_i$

This difference comes down to **inference** of $\hat{\beta}$ versus **prediction** of $\hat{y}_i$.

Inference vs Prediction

Prediction Problems:

1. How much will this ad campaign increase revenue?
2. What will traffic on the website be tomorrow?
3. Is this tweet harmful and should it be flagged?
4. What will the price of a home be in a year?

Inference Problems:

1. Will this ad campaign have a causal impact on revenue?
2. Does education have a causal effect on wages?
3. Will changing the Twitter UI cause people to spend more time on the site?
4. What is the causal effect of renovating a kitchen on a home price?

For more on the difference, check out this blog post by r y x, r

Types of Learning

- Supervised Learning:
  - You have a target variable ("dependent variable") and you would like to learn from function of the features ("independent variables") that explains the target.
- Unsupervised Learning:
  - We observe X, but not y. While we can't use traditional statistical models, we can still do things like "clustering," classify observations of X based on similarity.

There are two main types of supervised learning:

1. Regression: the target variable, y, is continuous and you want to learn a function f of the features X where $y = f(X) + \varepsilon$ where ε is some error term.

- If $\hat{f}$ is your estimate of f (or "learned function"), then the prediction of y, called $\hat{y}$ is $\hat{y} = \hat{f}(X)$

2. Classification: the target variable y is a category (e.g. $y =$ freshman, sophomore, junior, senior) and you want to learn $P(Y = y|X)$ so if you're given a new observation of X, you can predict which group it belongs to.

Types of Learning (cont.)

Below are some examples of each type you may have heard of.

- Supervised learning:
  - Regression*: linear regression, Ridge regression, LASSO regression.
  - Classification: logistic regression, K-nearest neighbors.
- Unsupervised learning: K-means, PCA.

* Don't confuse regression in the learning sense with regression in the statistical sense. While they are similar and have the same name, they are different. When we say linear regression, we are referring to estimating a condition mean $E[y|X]$ with a linear model. Regression in machine learning is any model where y is continuous regardless of what's being estimated.

Types of Learning (cont.)

Some "new" types of machine learning:

- **Deep Learning**: Use different versions and modifications of **Neural Network models** for different functions:
  - e.g. Feedforward Neural Networks, Transformers
- **Reinforcement Learning**: Learn from **interaction** with an environment to achieve a goal
  - e.g. Model-based RL such as AlphaGo

The resulting product of deep learning:

- **Large-Scale AI**:
  - e.g. Language models such as GPT-4, Claude, Gemini, Llama 3, Mistral, T5, BERT

Model Fitting

Disclaimer

Since this class is about learning programming skills for data analysis, I will skip many important and useful statistical concepts and special notices.

I am taking a **top-down** approach, talking about generally what we are doing and how we are achieving it with programming. If you are interested in the theory/math of each method and details/special concerns, please take:

- Econ 400 and Econ 470 in the economics department for econometrics.
- Classes in statistics department for general statistics theory.
- Linear algebra and numerical analysis class in math department for numerical optimization.

For this section, I will talk generally about data analysis with example of linear regression. However, there are many details about linear regressions are omitted.

Model Fitting

We have talked about data cleaning and data wrangling. What to do now with these cleaned data in hand?

For any type of problem, if you want the machine to solve it, it has to be converted to be a math problem. It means that you need three things:

1. **A mathematical model** to suit your problem and your data
2. **A concrete metric** to evaluate "success"
3. **An optimization method** to guide towards "success"

Think about the example of Google map, navigating to Gardner Hall (Your "Google Map" is actually an AI agent.):

1. **Model**: Map, the road people can walk on, distance (maybe average walking/driving speed and road condition, also other special rules)
2. **Metric**: Total Distance or Total time to get to Gardner Hall
3. **Optimization method**: Can be try every way and find the best (there are better ways which we will talk about).

Choose a Model

You should choose your model based on the feature of your data:

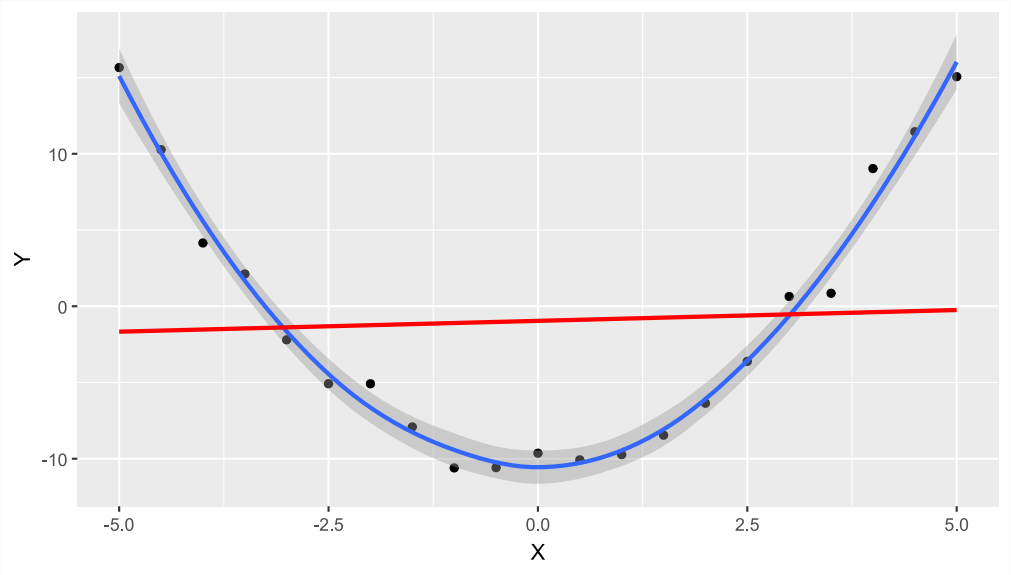
- If you have only X without y, you probably should do a unsupervised model.
- If you have X and y showing up as a line in a scatter plot, a linear regression model might be good enough.
- If you have X to be very weak signals of y for prediction, you probably need a more complex model (like tree-based models or neural network). etc.

Your model choice can also depends on the problem you try to solve:

- If you are doing prediction, you do not need to care about causality. Whatever model and variable drives good (out-of-sample) prediction is fine.
- If you are doing causal inference, you need to be really careful to include the correct variables, correct controls (sometimes, valid instruments) and correct functional form.

Model Example

Suppose that our data look like this:



We can see that we should definitely try to fit with a quadratic equation instead of a linear model here!

Be careful about the method you use for the data! Choose your model based on your data feature.

Also, be careful about interpretation of your model result!

Choose a Criteria to Optimize

The criteria to optimize also depends on your problem.

If you are trying to do prediction, you want the prediction error $e_i = Y_i - \hat{Y}_i$ to be as small as possible. How can we ensure that?

We should set our "goal" (objective function) to minimize the overall error, can we just try to minimize $\frac{1}{N}\sum_i e_i$?

No. The positive and negative error cancel out each other. Can we minimize absolute value, $\frac{1}{N}\sum_i |e_i|$?

Sure, but absolute value has a kink, which means that the math is more complicated.

One common practice would be to minimize $\frac{1}{N}\sum_i e_i^2$. You can see that this value is the mean of squared error, which we give a name as **"mean squared error" (MSE)**. If we are doing linear regression and we are minimizing the MSE, this method is usually called **"Ordinary Least Square" (OLS)**.

Other possible Criterias

There can be other objective function implemented. Here are two examples:

1. Likelihood Function: the likelihood of observing our current data, when assuming the data is from a certain distribution. The estimator from this objective function is called **"Maximum Likelihood Estimator" (MLE)**.

2. Mean Squared Moment/Expectation Error: We have a set of expectation conditions (moment conditions), maybe from economic theories and equations. We want the data fit as closely to the theory as possible. The method is often called **"Generalized Method of Moments" (GMM)**. (This is Nobel Price stuff!)

3. Have your own customized objective functions! For example, if you think that some points in the linear model is more important, you can weight the points by importance and minimize a weighted MSE, $\sum_i w_i * e_i^2$. This turns into Weighted Least Square.

Optimization

Sometimes, we are lucky that the model is simple enough that we can derive an analytical solution using math (calculus and linear algebra)!

Usually, if the function is nice (convex/concave and differentiable), we can take the derivative and the minimum/maximum is located at derivative = 0.

However, especially when we estimate complex models, the optimization math is not trivial. Numerical method is required and there are many different ways to do it. If you learn more about numerical method, you can write your own optimization algorithms. In R, one can simply use the function `optim`:

```
? optim
```

Notice that there are a lot of different methods can be used to find the optimized value, you can see from the documentation that "BFGS" method often works well.

The default functionality of `optim` is to minimize the objective function.

- Fact: Maximizing f is the same thing as minimizing $-f$.

So, if need to maximize some objective function f, you can do `optim(par, -fn)`.

Optimization Example

Suppose that we are trying to *minimize* an objective function $f(x) = (3x - 4)^2$. We need to create the function and set the initial value (often, the initial value matters). I will use BFGS method here.

```
# Create the function first
funx = function(x) {return((3*x - 4)^2)}

# Set the initial value. Usually, it is better to have an educated guess
Initial_value = 0

# Use optim to find minimum
optim(Initial_value, funx, method = "BFGS")
```

```
## $par
## [1] 1.333333
##
## $value
## [1] 1.972152e-29
##
## $counts
## function gradient
##       10        3
##
## $convergence
```

Model Fitting: Simple Linear Regression

Model Fitting with Linear Regression

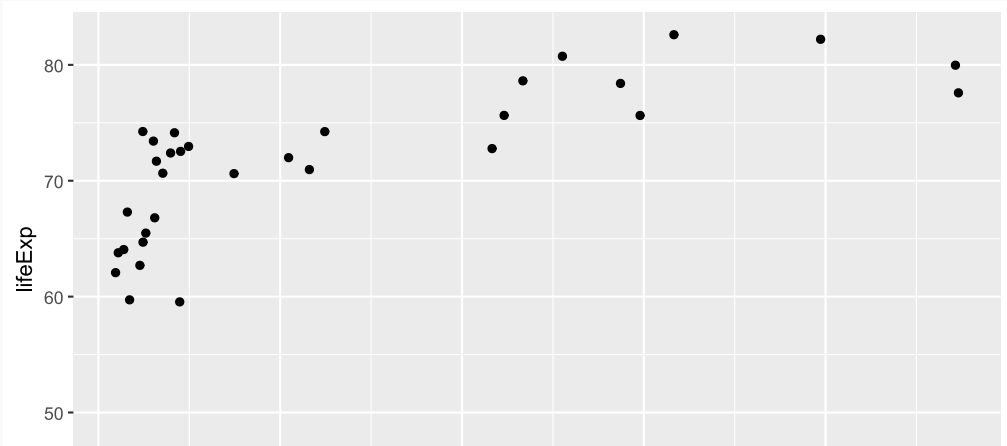
Suppose that our question to answer is:

- How GDP per capita is related to life expectancy?

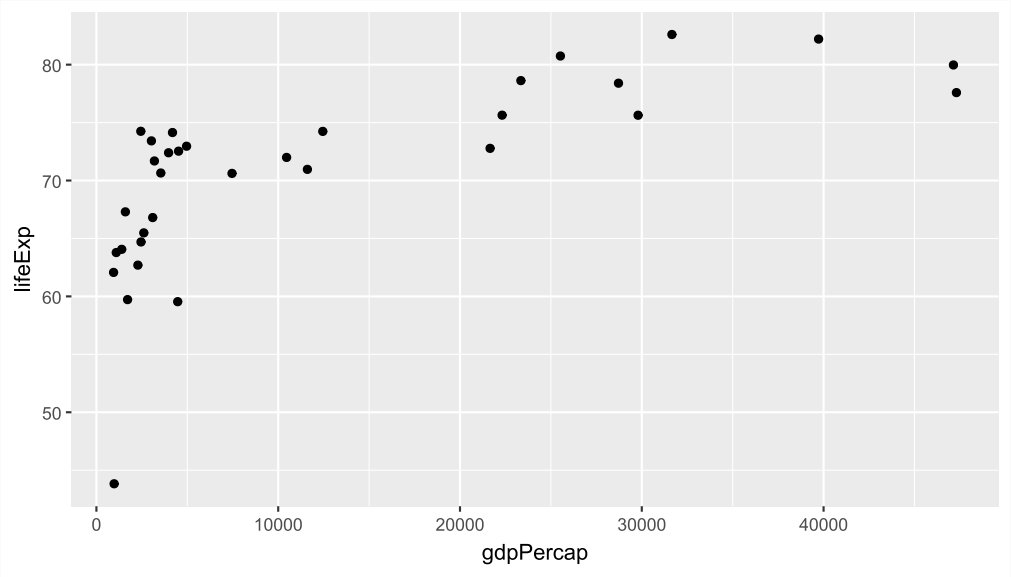
Before deciding any model, we should look into the data.

Let's go back to our `gapminder` dataset.

```
library(gapminder)
gp_subset = gapminder[gapminder$continent=="Asia"&gapminder$year==2007,]
g = ggplot(gp_subset,aes(x=gdpPercap, y=lifeExp)) +
  geom_point()
g
```



We've seen an example already...

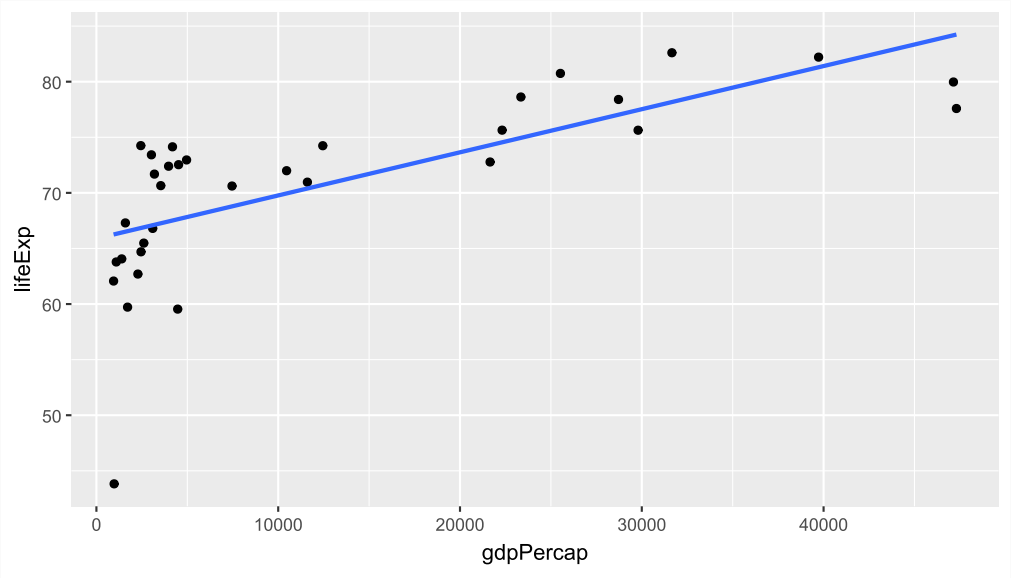


How can we assess the relationship between GDP per capita and life expectancy? If a country in Asia in 2007 had a GDP per capita of 15,000, what would we expect its life expectancy to be?

We've seen an example already...

In our first class, we added a line of best fit:

```
g + geom_smooth(method='lm',se=FALSE)
```



So: how would we characterize the relationship? If a country in Asia in 2007 had a GDP per capita of 15,000, what would we expect its life expectancy to be?

Linear Model

The relationship is not too far from a straight line. Let's use a linear model:

$$LifeExp_i = b_0 + b_1 GDPpc_i + e_i$$

Now, let's select our objective function. MSE is good enough and intuitive so let's use it here!

$$MSE = \frac{1}{N} \sum_i e_i^2$$

Some linear model and consequences

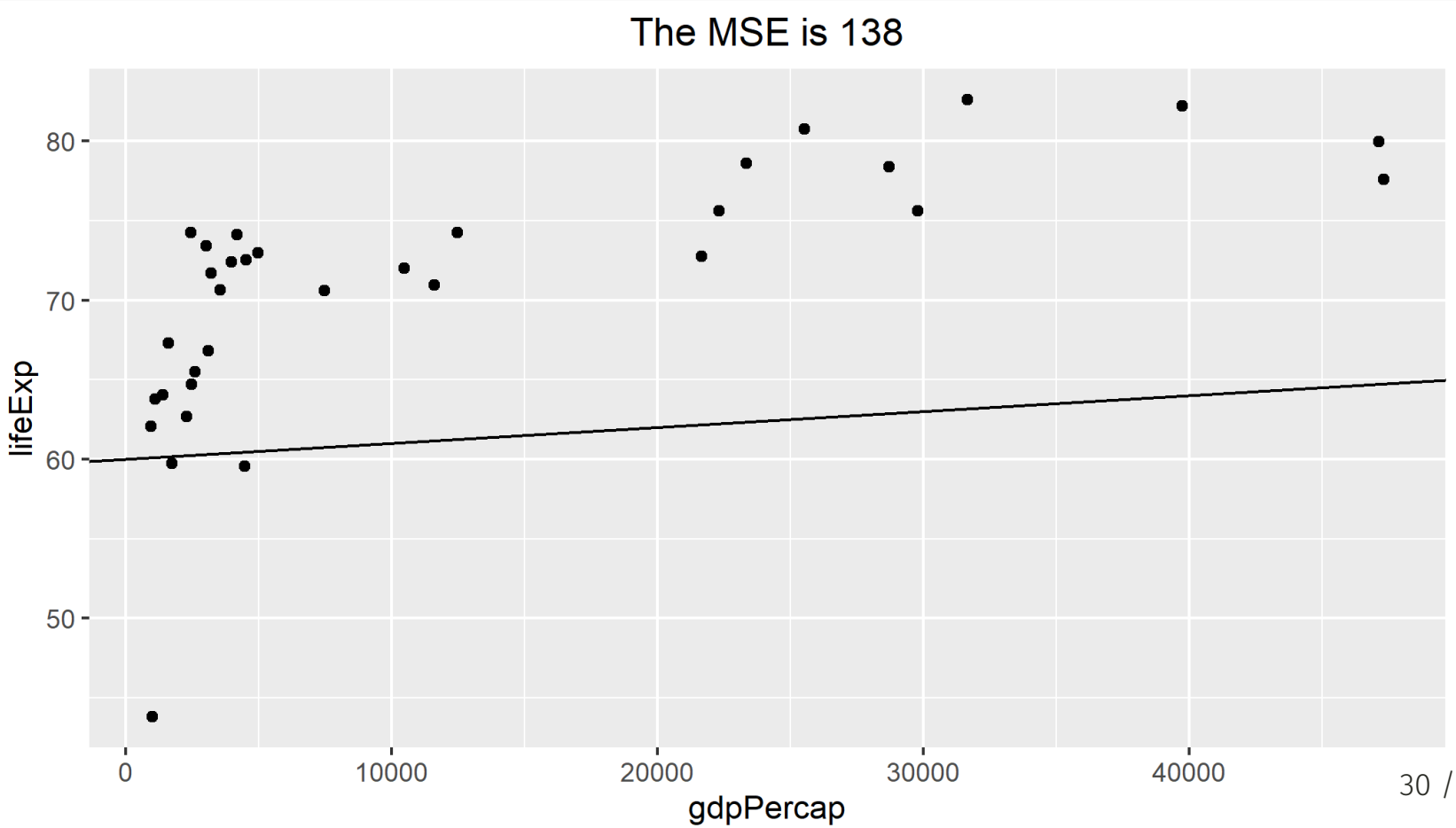
I have created a function for trying different values of b_0 and b_0 and see the consequences. It is in the template.

```
# Build a MSE function for gp_subset
MSE = function(b, Graph = F) {
  # b is a vector containing b0 and b1
  e = gp_subset$lifeExp - b[1] - b[2]*gp_subset$gdpPercap

  # plot
  if (Graph == T){
    (ggplot(gp_subset,aes(x=gdpPercap, y=lifeExp)) +
       geom_point() +
       geom_abline(slope = b[2], intercept = b[1]) +
       labs(title = paste("The MSE is", round(mean(e^2)))) +
       theme(plot.title = element_text(hjust = 0.5))) ▷
      print()
  }

  return(mean(e^2))
}
```

Try some values!

Here, let's try $b_0 = 60$ and $b_1 = 0.0001$:

```
MSE(b = c(60, 0.0001), Graph = T)
```



Optimize

Now, we have the MSE function, how about use numerical method to find the best b_0 and b_1 forming line with minimized MSE. Let's use $b_0 = 60$ and $b_1 = 0.0001$ as initial values:

```
# Use optim to find best b0 and b1
optim(c(60, 0.0001), MSE)
```

```
## $par
## [1] 6.589298e+01 3.877727e-04
##
## $value
## [1] 32.27373
##
## $counts
## function gradient
##       99       NA
##
## $convergence
## [1] 0
##
## $message
## NULL
```

Linear Regression in R

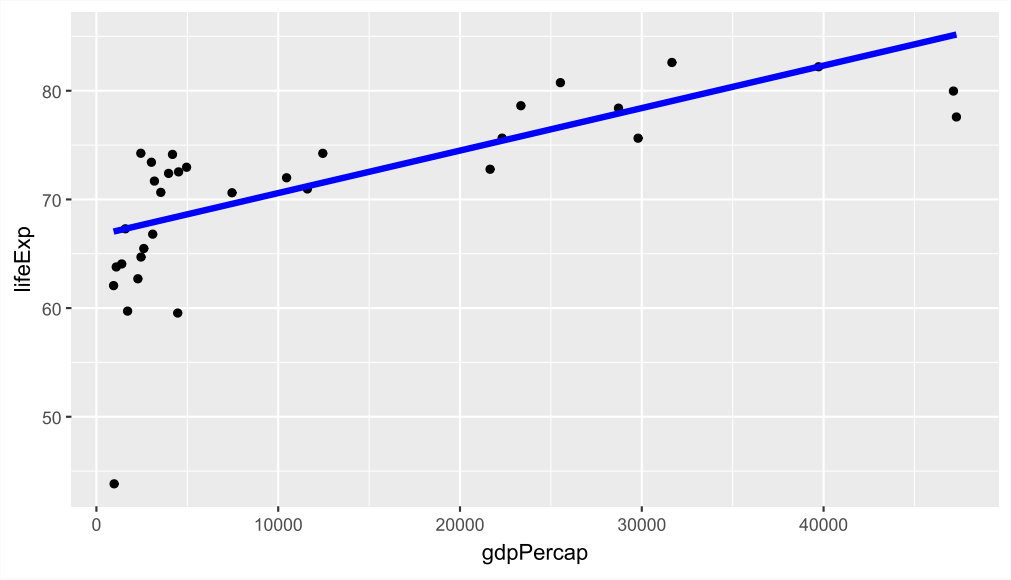
Luckily, we have a built-in function for OLS in R! Try `lm` and see whether you get the same result:

```
Result = lm(lifeExp~gdpPercap, data=gp_subset)
summary(Result)
```

```
##
## Call:
## lm(formula = lifeExp ~ gdpPercap, data = gp_subset)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22.4409  -2.3685   0.9101   4.4345   7.4111
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 6.589e+01  1.369e+00  48.123  < 2e-16 ***
## gdpPercap   3.878e-04  7.320e-05   5.298 9.13e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.861 on 31 degrees of freedom
## Multiple R-squared:  0.4752,    Adjusted R-squared:  0.4583
```

Other Options Can Work!

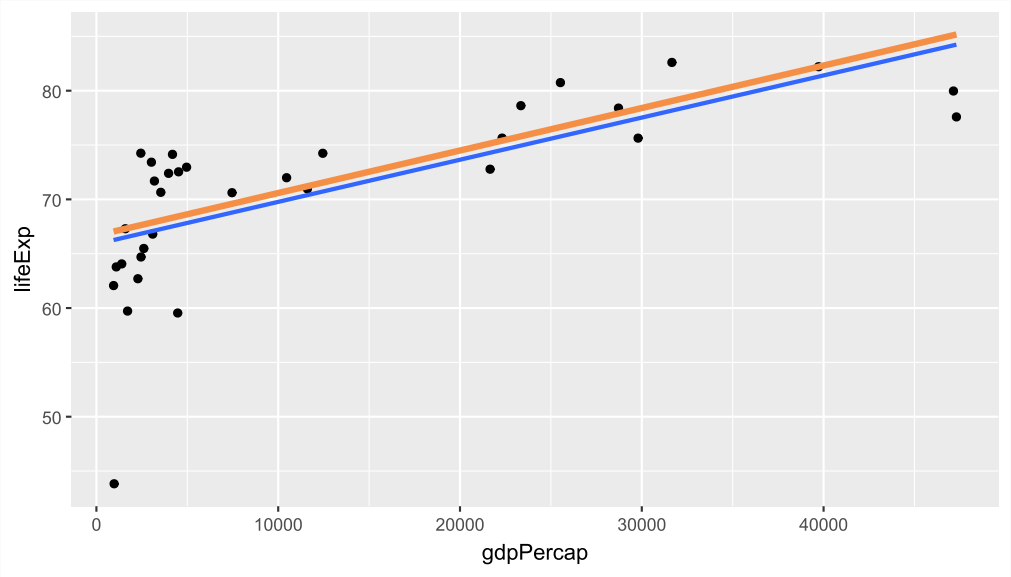
We could just as easily minimize the ***absolute deviations***. This is called Least absolute deviations regression (LAD).



This line looks a lot better than our ad hoc lines! We could certainly use it to assess relationships or predict life expectancy.

We typically don't though. OLS has some *very* nice properties, and so it's become the first thing in every data scientist's toolkit.

In Case You Were Curious:



Here's the difference between minimizing *absolute* deviations and minimizing ***squared*** deviations.

Other Models

Multiple Linear Regression

Suppose that we want to see multiple variable's effect on LifeExp:

$$LifeExp_{i,t} = b_0 + b_1 * GDPpc_{i,t} + b_2 * Year_t$$

We can simply use `lm` again:

```
Result = lm(lifeExp~gdpPercap + year, data=gapminder)
summary(Result)
```

```
##
## Call:
## lm(formula = lifeExp ~ gdpPercap + year, data = gapminder)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -67.262  -6.954   1.219   7.759  19.553
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -4.184e+02  2.762e+01  -15.15   <2e-16 ***
## gdpPercap    6.697e-04  2.447e-05   27.37   <2e-16 ***
## year         2.390e-01  1.397e-02   17.11   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Polynomial Regressions

There could be non-linear effects, we can see that GDPpc has a slightly non-linear effect:

$$LifeExp_i = b_0 + b_1 * GDPpc_i + b_2 * GDPpc_i^2$$

Do we need a new R function?

```
# Create new squared variable and run lm
Result = gp_subset ▷
  mutate(GDPpc_sq = gdpPercap^2) ▷
  lm(lifeExp~gdpPercap + GDPpc_sq, data=_)
summary(Result)
```

```
##
## Call:
## lm(formula = lifeExp ~ gdpPercap + GDPpc_sq, data = mutate(gp_subset,
##     GDPpc_sq = gdpPercap^2))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -20.7425  -1.7516   0.3214   2.5116   8.3800
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
```

Log Regression

We can also use the same trick to have y or x to be in log:

```
# Create new log variable and run lm
Result = gp_subset ▷
  mutate(log_GDPpc = log(gdpPercap)) ▷
  lm(lifeExp~log_GDPpc, data=_)
summary(Result)
```

```
##
## Call:
## lm(formula = lifeExp ~ log_GDPpc, data = mutate(gp_subset, log_GDPpc = log(gdpPercap)))
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -17.314  -1.650  -0.040   3.428   8.370
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  25.6501     6.1234   4.189 0.000216 ***
## log_GDPpc     5.1573     0.6939   7.433 2.26e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.851 on 31 degrees of freedom
```

Ridge, LASSO and Elastic Net

Sometimes, we have too many variables and only want to keep the "useful" ones. For example, we want to use the `mtcars` data, 'mpg', 'wt', 'drat', 'qsec' to predict 'hp' (Note that there are not too many variables here. This is rather a demonstration).

There are three methods can be done by manipulate the objective function:

- Ridge: \sum_i e_i^2 + \lambda \sum_k (b_k)^2
- LASSO: \sum_i e_i^2 + \lambda \sum_k |\beta|
- Elastic Net: \sum_i e_i^2 + \lambda ((1-\alpha)/2 \sum_k (b_k)^2 + \alpha \sum_k |\beta|)

Note that if $\alpha = 1$ in Elastic Net, it becomes LASSO. If $\alpha = 0$ in Elastic Net, it becomes Ridge.

We can use `glmnet` function in `glmnet` package. This is a Elastic Net function, but we can change α to make it Ridge or Lasso.

Ridge Example

We will just use $\alpha = 0$ to perform Ridge, but we need to set an λ. Something like λ is called tuning parameter or hyper parameter in machine learning. We will talk about how to choose it carefully in the lecture 10.

Meanwhile, let's choose $\lambda = 1$. What would be the effect if you increase it? (Try to change it to 100, 1000, etc.)

```
library(glmnet)
#define response variable
y ← mtcars$hp

#define matrix of predictor variables
x ← data.matrix(mtcars[, c('mpg', 'wt', 'drat', 'qsec')])

coef(glmnet(x, y, alpha= 0, lambda = 1))
```

```
## 5 x 1 sparse Matrix of class "dgCMatrix"
##                      s0
## (Intercept) 476.565783
## mpg          -3.022045
## wt           24.723220
## drat          4.034159
## qsec        -20.349515
```

LASSO Example

We will just use $\alpha = 1$ to perform LASSO, it is the default argument.

Meanwhile, let's choose $\lambda = 1$. What would be the effect if you increase it? (Try to change it to 100, 1000, etc.)

Note that the variable is dropped if there is a dot (coefficient 0).

```
coef(glmnet(x, y, lambda = 1))
```

```
## 5 x 1 sparse Matrix of class "dgCMatrix"
##                      s0
## (Intercept) 490.818087
## mpg          -2.803152
## wt           23.635921
## drat          .
## qsec        -20.385527
```

Logistic Regression

Suppose we want to predict the transmission type (am) (0 = automatic, 1 = manual) based on a few predictors like mpg and hp. This becomes a classification problem and we can use logistic regression.

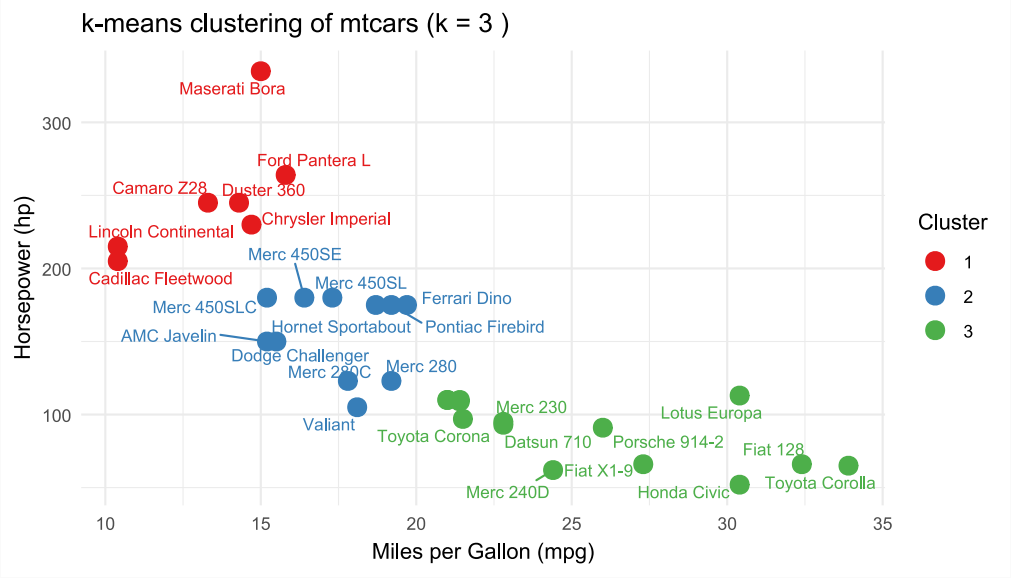
```
# Convert 'am' to factor
mtcars$am ← factor(mtcars$am, labels = c("Automatic", "Manual"))

# Fit logistic regression
# Predict transmission based on mpg and hp
Result ← glm(am ~ mpg + hp, data = mtcars, family = binomial)

# View summary
summary(Result)
```

```
##
## Call:
## glm(formula = am ~ mpg + hp, family = binomial, data = mtcars)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -33.60517   15.07672  -2.229   0.0258 *
## mpg           1.25961    0.56747   2.220   0.0264 *
## hp            0.05504    0.02692   2.045   0.0409 *
## ---
```

K-Means Clustering

K-Means Clustering is a type of unsupervised learning to group observations based on their features.

Suppose we want to see what cars are similar in `mtcars` but there is no label. We want to group the similar cars into K groups.



Next lecture(s): Model Tunning and Testing